

Project Title	SYSTEM ARCHITECTURE AND DESIGN DOCUMENT
Student Name	KANJO ELKAMIRA NDI
Matriculation Number	UBa/YIBSSE25P00300
Programme	Master's of Technology in Software Engineering
Course Code	CENY6203
Date of Submission	June 1st 2026
Course Coordinator	Prof. Yongho Louis

TABLE OF CONTENTS

1. C4 ARCHITECTURE OVERVIEW.....	3
1.2 C2 - Containers: deployable units.....	4
1.3 C3 - Components: Express API internals.....	6
2. DATABASE SCHEMA DESIGN.....	8
2.1 Entities and relationships.....	8
2.2 Indexing strategy.....	9
2.3 Prisma schema key excerpts.....	11
3. API SPECIFICATION (OPENAPI 3.0 SUMMARY).....	13
3.1 Auth endpoints (base: /api/v1/auth) : students & posters via Better Auth.....	14
3.2 Super Admin auth endpoints (base: /api/v1/superadmin/auth).....	14
3.3 Gigs endpoints (base: /api/v1/gigs).....	15
3.4 Applications endpoints (base: /api/v1).....	16
3.5 Reviews, Users and light-Admin endpoints.....	16
3.6 Super Admin endpoints (base: /api/v1/superadmin).....	17
.....	19
4. PROJECT FOLDER STRUCTURE AND CODING CONVENTIONS.....	20
4.1 Repository root (monorepo).....	20
4.2 Backend structure.....	21
4.3 Admin Panel structure.....	22
4.4 Frontend structure.....	23
4.5 Coding conventions.....	23
5. GIT REPOSITORY AND BRANCHING STRATEGY.....	26

5.1 Branching model.....	26
5.2 Commit message standard (Conventional Commits).....	27
5.3 Pull request checklist.....	27
5.4 GitHub repository requirements.....	27
5.5 Environment variables.....	28

TABLE OF FIGURES

Figure 1 : Context Diagram.....	4
Figure 2 : Container Diagram.....	5
Figure 3 : Component Diagram.....	8
Figure 4 : ER-Diagram.....	13
Figure 5 : Auth Flow Diagram.....	19
Figure 6 : Deployment Architecture.....	25

1. C4 ARCHITECTURE OVERVIEW

The Campus Gigs system is described using the C4 model. The diagrams in the submission package correspond to the levels documented below.

1.1 C1 - Context: actors and external systems

Actor / System	Type	Role / Integration
Gig Worker (student)	<i>Person</i>	Browses gig feed, applies, messages posters, receives reviews
Gig Poster (student)	<i>Person</i>	Posts gigs, reviews applicants, hires, marks gigs complete
Super Admin	<i>Person</i>	Full platform oversight: user management, gig moderation, analytics, university configuration, ban/unban. Authenticates via dedicated credentials flow — completely separate from Better Auth.
Better Auth	<i>External SaaS</i>	Social OAuth (Google), JWT sessions, RBAC for student/poster roles
Resend	<i>External SaaS</i>	Transactional email: alerts, review requests, notifications
Cloudinary	<i>External SaaS</i>	Image and file storage for gig attachments and avatars

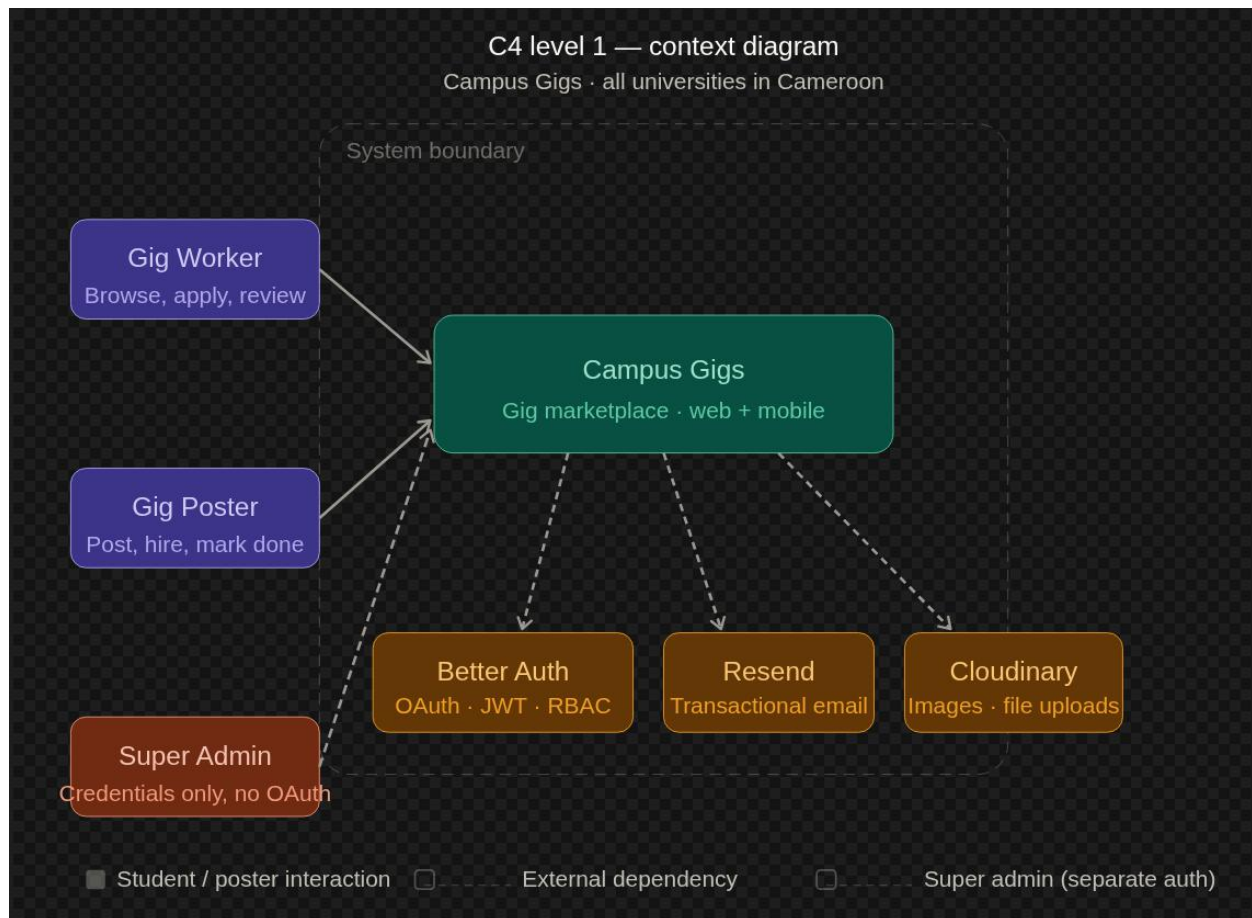


Figure 1: Context Diagram

1.2 C2 - Containers: deployable units

Container	Tech stack	Deployment target
React Web App	<i>Vite + React 18 + TS + Tailwind + shadcn/ui + Framer Motion</i>	Vercel (auto-deploy on main)
Express REST API	<i>Node.js + Express + TypeScript + Better Auth + Socket.io</i>	Render.com or Railway (Node and DB service)
Redis	<i>ioredis client</i>	Render Redis or Upstash free tier

Flutter Mobile	<i>Flutter 3 + Dart, same REST API</i>	APK for demo; EAS Build for distribution
Admin Panel SPA	<i>Vite + React + TypeScript + Tailwind + shadcn/ui (separate app, no Better Auth)</i>	Vercel (separate deployment, admin subdomain)

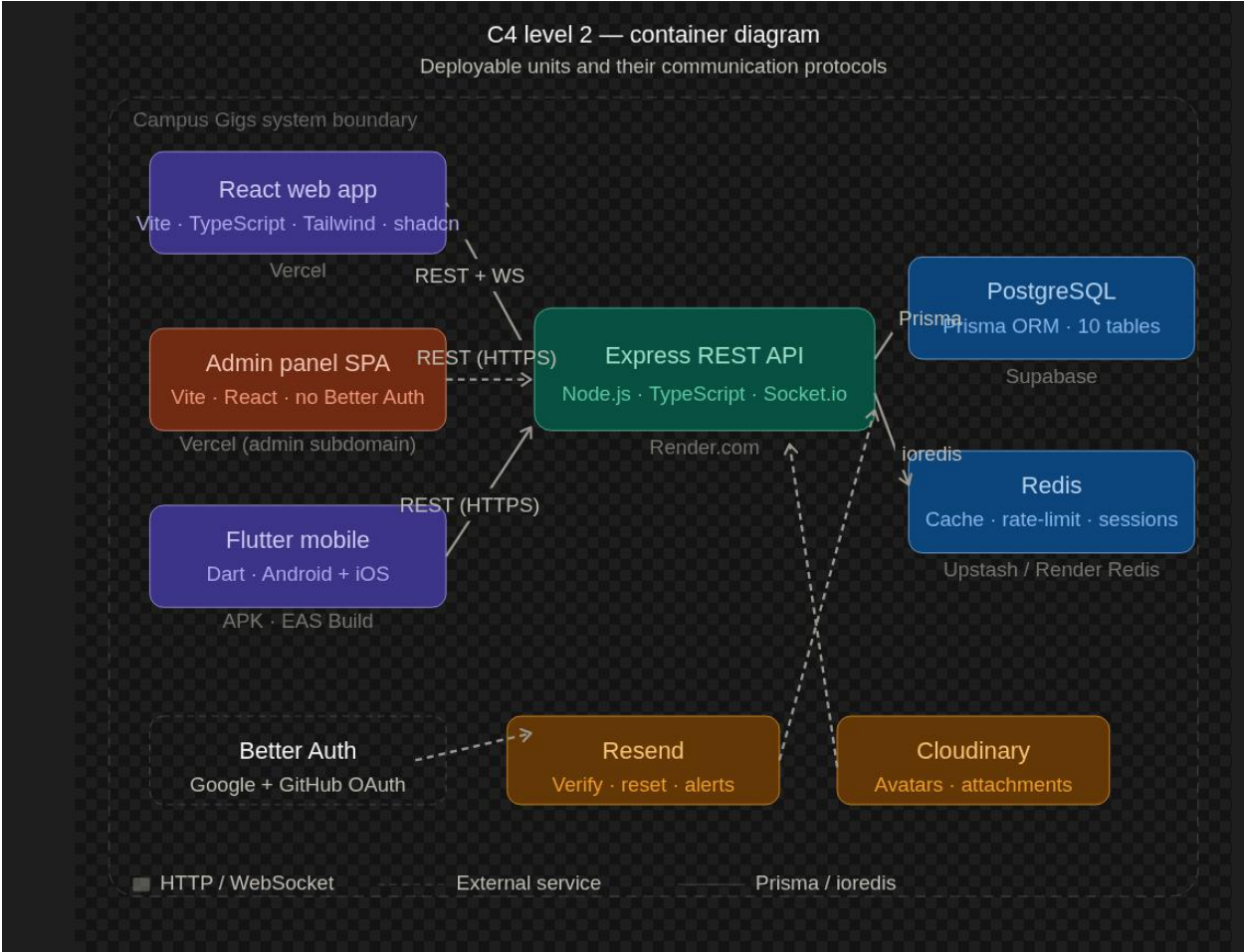


Figure 2: Container Diagram

1.3 C3 - Components: Express API internals

The API follows: Routes -> Controllers -> Services -> Data Access (Prisma / Redis).

Module	Layer	Responsibility
Middleware pipeline	<i>Cross-cutting</i>	Helmet, CORS, rate-limiter, Better Auth guard, Zod validator, Pino logger, error handler
Auth router	<i>Route/Controller</i>	register, login, logout, /me, OAuth callbacks (Better Auth — students/posters only)
Super Admin Auth router	<i>Route/Controller</i>	POST /superadmin/auth/login — credentials-only (email+password, bcrypt). Returns signed superadmin JWT with separate secret. No OAuth, no social login, no Better Auth.
Gigs router	<i>Route/Controller</i>	Full CRUD, search, filter, pagination
Applications router	<i>Route/Controller</i>	apply, accept/reject, status lifecycle
Reviews router	<i>Route/Controller</i>	create and list bidirectional reviews
Users router	<i>Route/Controller</i>	profile read/update, public profiles
Admin router (ADMIN role)	<i>Route/Controller</i>	Light moderation by promoted users (ban single user, close gig)
Super Admin router	<i>Route/Controller</i>	Full platform management: user CRUD, bulk actions, university config, platform stats, audit log, content moderation queue, system health. Protected by

		superadminGuard middleware.
Token service	<i>Service</i>	Better Auth session management, JWT signing for student/poster tokens
Super Admin Token service	<i>Service</i>	Separate JWT signing with SUPERADMIN_JWT_SECRET env var. Tokens expire in 4 hours. Stored in httpOnly cookie. Refresh via re-login only (no refresh token).
Email service	<i>Service</i>	Resend SDK: templates for alerts and notifications
Upload service	<i>Service</i>	Cloudinary SDK: image upload and CDN URL generation
Chat service	<i>Service</i>	Socket.io room-based messaging per application thread
Prisma client	<i>Data access</i>	Type-safe queries, migrations, connection pooling
Redis client	<i>Data access</i>	ioredis: feed cache (60 s TTL), rate-limit counters

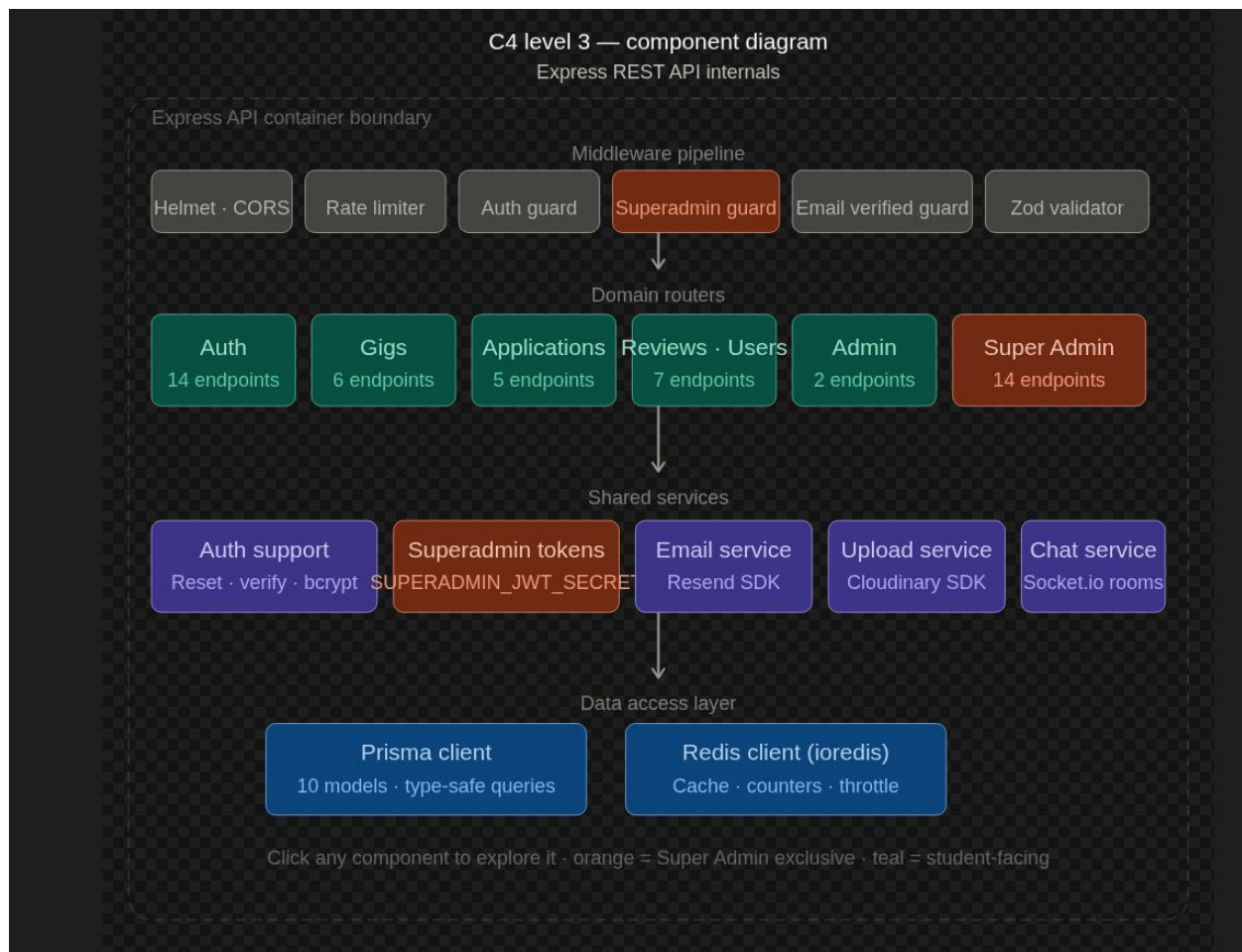


Figure 3: Component Diagram

2. DATABASE SCHEMA DESIGN

PostgreSQL is the primary data store managed through Prisma migrations. The schema is in 3NF with UUID primary keys throughout.

2.1 Entities and relationships

Table	Cardinality	Key relationships
users	<i>Central entity</i>	1-to-many: gigs (poster), applications (worker), reviews (reviewer + reviewee), messages
categories	<i>Lookup / seed data</i>	1-to-many: gigs (every gig belongs to one category)

gigs	<i>Core entity</i>	belongs-to: users (poster), categories; 1-to-many: applications, reviews
applications	<i>Junction entity</i>	belongs-to: gigs and users (worker); 1-to-many: messages
reviews	<i>Outcome entity</i>	belongs-to: gigs, reviewer, reviewee; unique(gig_id, reviewer_id) prevents duplication
messages	<i>Chat entity</i>	belongs-to: applications (thread) and sender (user); ordered by sent_at
super_admins	<i>Separate auth entity</i>	Completely isolated from the users table. Stores hashed credentials and session metadata. No FK to users table.
audit_logs	<i>Immutable log</i>	Records every super admin action: who, what, target entity, timestamp, IP address. Append-only, never deleted.

2.2 Indexing strategy

Table.column(s)	Index type	Query pattern
gigs.poster_id	<i>B-tree</i>	Poster dashboard: load all my gigs
gigs.category_id	<i>B-tree</i>	Feed filter by category
gigs.status	<i>B-tree</i>	Filter open / closed / completed
gigs.created_at	<i>B-tree DESC</i>	Default newest-first feed sort

gigs.title (GIN)	<i>Full-text tsvector</i>	Keyword search on title and description
applications.gig_id	<i>B-tree</i>	Load all applicants for a gig
applications.worker_id	<i>B-tree</i>	Worker: load all gigs I applied to
reviews.reviewee_id	<i>B-tree</i>	Profile: load all reviews received
messages.application_id	<i>B-tree</i>	Load chat thread per application
users.email	<i>Unique B-tree</i>	Auth lookup; enforces uniqueness
super_admins.email	<i>Unique B-tree</i>	Super admin login lookup
audit_logs.created_at	<i>B-tree DESC</i>	Audit log pagination, newest first
audit_logs.admin_id	<i>B-tree</i>	Filter audit log by which admin acted

2.3 Prisma schema key excerpts

Full schema at `/backend/prisma/schema.prisma`. Core models shown below:

```
model User {
  id          String  @id @default(uuid())
  email       String  @unique
  fullName    String
  avatarUrl   String?
  role        UserRole @default(WORKER)
  universityId String
  avgRating   Float   @default(0)
  reviewCount Int     @default(0)
  isBanned    Boolean @default(false)
  createdAt   DateTime @default(now())
  gigs        Gig[]   @relation('PosterGigs')
  applications Application[]
  reviewsGiven Review[] @relation('ReviewerReviews')
  reviewsRecvd Review[] @relation('RevieweeReviews')
  messages    Message[]
}
```

// NEW — completely separate from users table

```
model SuperAdmin {
  id          String  @id @default(uuid())
  email       String  @unique
  passwordHash String  // bcrypt, cost factor 12
  fullName    String
  lastLoginAt DateTime?
  lastLoginIp String?
  createdAt   DateTime @default(now())
  auditLogs   AuditLog[]
}
```

// NEW — immutable action log

```
model AuditLog {
  id          String  @id @default(uuid())
```

```
adminId    String
admin      SuperAdmin @relation(fields:[adminId], references:[id])
action     String    // e.g. BAN_USER, DELETE_GIG, PROMOTE_ADMIN
targetType String    // 'user' | 'gig' | 'review' | 'university'
targetId   String
meta       Json?     // snapshot of changed fields
ipAddress  String
createdAt  DateTime @default(now())
@@index([adminId])
@@index([createdAt(sort:Desc)])
}
```

```
enum UserRole { WORKER POSTER ADMIN }
```

```
enum GigStatus { OPEN IN_PROGRESS COMPLETED CANCELLED }
```

```
enum AppStatus { PENDING ACCEPTED REJECTED COMPLETED }
```

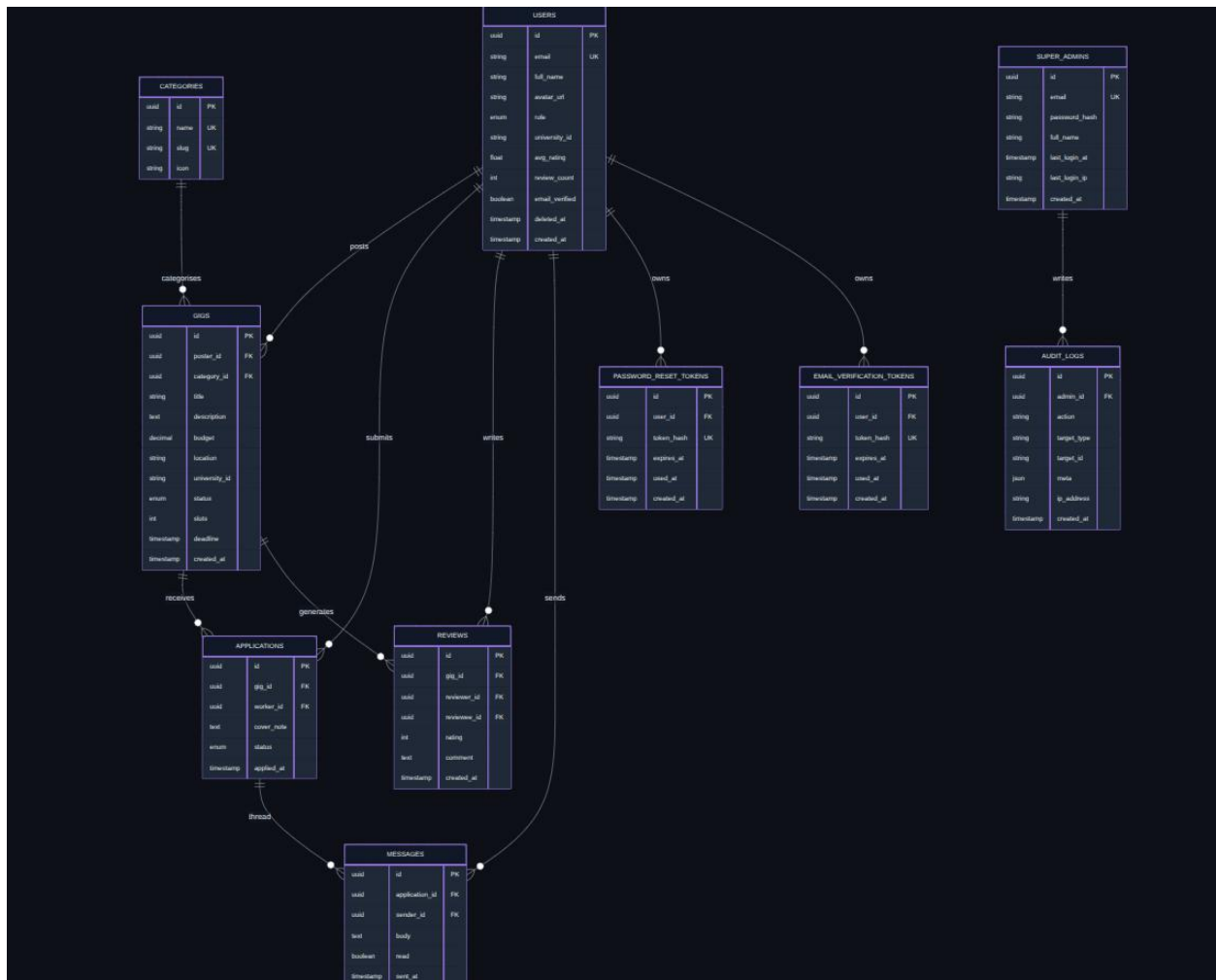


Figure 4: ER-Diagram

3. API SPECIFICATION (OPENAPI 3.0 SUMMARY)

The REST API exposes 27 endpoints across 6 domains (20 original + 7 new super admin endpoints). Full OpenAPI YAML auto-generated via swagger-jsdoc at /api/docs. Base URL: /api/v1.

3.1 Auth endpoints (base: /api/v1/auth) : students & posters via Better Auth

Method	Endpoint	Auth?	Description
POST	/register	None	Create student/poster account. Returns JWT via Better Auth.
POST	/login	None	Email/password login via Better Auth. Returns JWT + refresh token.
GET	/google	None	Redirect to Google OAuth (Better Auth handler).
POST	/logout	Bearer	Invalidate Better Auth session.
GET	/me	Bearer	Return authenticated student/poster profile.
POST	/refresh	Refresh token	Issue new JWT from valid refresh token.

3.2 Super Admin auth endpoints (base: /api/v1/superadmin/auth)

Method	Endpoint	Auth?	Description
POST	/login	None	Credentials-only login. Body: { email, password }. Verifies against super_admins table with bcrypt. Returns signed JWT using SUPERADMIN_JWT_SECRET (separate from student JWT secret). Sets httpOnly cookie. No OAuth, no social login.

POST	/logout	SuperAdmin JWT	Clears httpOnly session cookie.
GET	/me	SuperAdmin JWT	Returns super admin profile and last-login metadata.

3.3 Gigs endpoints (base: /api/v1/gigs)

Method	Endpoint	Auth?	Description
GET	/	None	List open gigs. Query: ?q= &category= &university= &minBudget= &maxBudget= &page= &limit=
POST	/	Bearer (POSTER)	Create gig. Body: title, description, budget, location, categoryId, slots, deadline.
GET	/:id	None	Get single gig with poster profile and application count.
PATCH	/:id	Bearer (owner)	Update gig fields. Blocked after first acceptance.
DELETE	/:id	Bearer (owner)	Soft-delete: sets status=CANCELLED, notifies applicants.
PATCH	/:id/close	Bearer (owner)	Mark gig as COMPLETED.

3.4 Applications endpoints (base: /api/v1)

Method	Endpoint	Auth?	Description
GET	/gigs/:id/applications	Bearer (poster)	List all applications for a gig.
POST	/gigs/:id/applications	Bearer (WORKER)	Apply to gig. Body: coverNote. Prevents duplicates.
PATCH	/applications/:id/accept	Bearer (poster)	Accept application. Auto-rejects others if slots filled.
PATCH	/applications/:id/reject	Bearer (poster)	Reject application. Notifies worker by email.
PATCH	/applications/:id/complete	Bearer (poster)	Mark completed. Triggers review prompt for both parties.

3.5 Reviews, Users and light-Admin endpoints

Method	Endpoint	Auth?	Description
POST	/reviews	Bearer	Create review. Body: gigId, revieweeId, rating (1-5), comment.
GET	/users/:id/reviews	None	All reviews received by user. Includes avg rating.
GET	/users/:id	None	Public profile: name, avatar, rating, active gigs.
PATCH	/users/me	Bearer	Update own profile: name, avatar, university.

GET	/categories	None	List all gig categories.
GET	/admin/users	Bearer (ADMIN)	Light admin: list users.
PATCH	/admin/users/:id/ban	Bearer (ADMIN)	Light admin: ban a user account.

3.6 Super Admin endpoints (base: /api/v1/superadmin)

All routes protected by superadminGuard middleware verify JWT signed with SUPERADMIN_JWT_SECRET. Every mutating action writes to the audit_log table.

Method	Endpoint	Auth?	Description
GET	/dashboard	SuperAdmin JWT	Platform stats: total users, gigs, applications, revenue, universities active, daily active chart data.
GET	/users	SuperAdmin JWT	Full user list with filters: role, university, banned status, date range. Paginated.
PATCH	/users/:id	SuperAdmin JWT	Update any user field. Ban/unban, change role, verify identity.
DELETE	/users/:id	SuperAdmin JWT	Hard delete user and all their data. Irreversible. Writes detailed audit entry.
GET	/gigs	SuperAdmin JWT	All gigs across all statuses and universities. Full filter set.

DELETE	/gigs/:id	SuperAdmin JWT	Hard delete a gig and its applications. Notifies all parties.
GET	/audit-logs	SuperAdmin JWT	Paginated audit log. Filter by admin, action type, date range.
GET	/universities	SuperAdmin JWT	List all configured universities with gig and user counts.
POST	/universities	SuperAdmin JWT	Add a new university to the platform.
DELETE	/reviews/:id	SuperAdmin JWT	Remove a review that violates community guidelines.

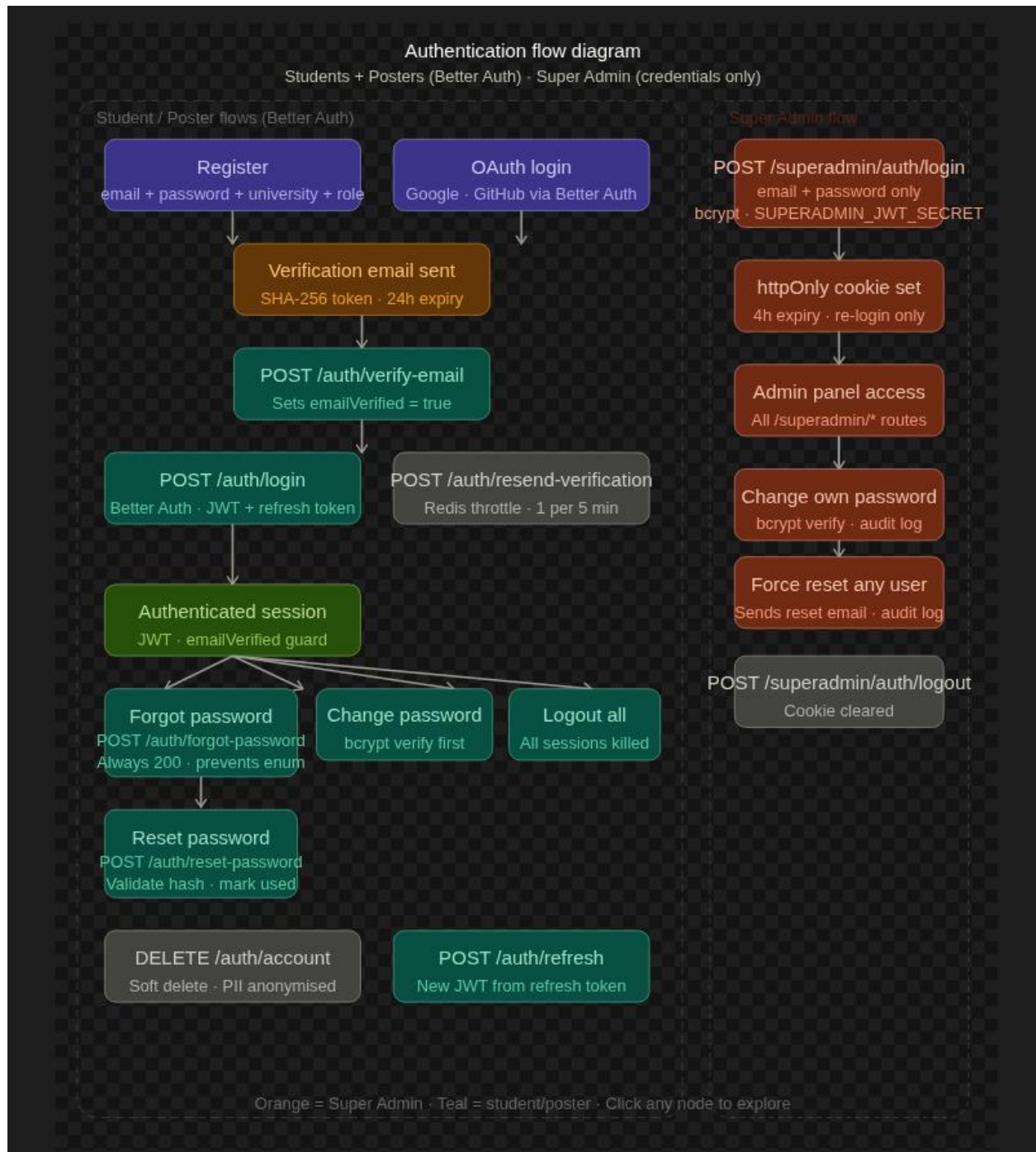


Figure 5: Auth Flow Diagram

4. PROJECT FOLDER STRUCTURE AND CODING CONVENTIONS

4.1 Repository root (monorepo)

```
campus-gigs/  
|- backend/      # Express API (Node.js + TypeScript)  
|- frontend/     # React web app — students & posters (Vite +  
TypeScript)  
|- admin-panel/  # NEW: Super Admin SPA (Vite + React, separate  
app)  
|- mobile/       # Flutter app (Dart)  
|- docs/         # Architecture docs, wireframes, API spec  
|- .github/  
|  ` - workflows/ # GitHub Actions CI/CD  
|- docker-compose.yml # Local: postgres + redis + api + web +  
admin  
|- .gitignore  
` - README.md
```

4.2 Backend structure

```
backend/src/
|- config/          # env validation, db config, redis config
|- middleware/
|  |- authGuard.ts  # Better Auth guard (students/posters)
|  |- superadminGuard.ts # NEW: verifies SUPERADMIN_JWT_SECRET token
|  |- validator.ts
|  |- errorHandler.ts
|  `-- logger.ts
|- modules/
|  |- auth/          # Better Auth: controller, service, routes, dto
|  |- gigs/
|  |- applications/
|  |- reviews/
|  |- users/
|  |- admin/         # Light admin (ADMIN role users)
|  `-- superadmin/   # NEW: controller, service, routes for super admin
|      |- superadmin.controller.ts
|      |- superadmin.service.ts
|      |- superadmin.auth.ts  # credentials login, bcrypt verify, JWT sign
|      `-- superadmin.routes.ts
|- services/         # email, upload, chat (shared singletons)
|- lib/              # prisma.ts, redis.ts, socket.ts
|- utils/            # pagination, slugify, asyncWrapper
`- app.ts            # Express factory
```

```
backend/
|- prisma/schema.prisma # includes SuperAdmin + AuditLog models
|- prisma/migrations/
|- prisma/seed.ts       # seeds default super admin (env var credentials)
|- tests/unit/
|- tests/integration/
`- Dockerfile
```

4.3 Admin Panel structure

admin-panel/ is a completely separate Vite + React application.

It has its own package.json, its own Tailwind config, and its own deployment.

It is NEVER imported by frontend/. The two apps share only TypeScript types via a shared types file.

admin-panel/

```
| - src/
|   | - components/
|   |   | - layout/
|   |   |   | - AdminNavbar.tsx    # top nav with admin name + logout
|   |   |   | - AdminSidebar.tsx  # full sidebar nav
|   |   |   | - AdminLayout.tsx   # wraps all admin pages
|   |   | - tables/               # DataTable, sortable columns, pagination
|   |   | - charts/               # recharts wrappers: LineChart, BarChart
|   |   | - shared/
|   |   |   | - ConfirmModal.tsx  # destructive action confirmation
|   |   |   | - AuditLogRow.tsx   # single audit log entry display
|   |   |   | - StatCard.tsx      # dashboard metric card
|   | - pages/
|   |   | - AdminLogin.tsx        # credentials form, no Google/OAuth
|   |   | - Dashboard.tsx        # platform stats + charts
|   |   | - Users.tsx             # user management table
|   |   | - UserDetail.tsx        # single user with action buttons
|   |   | - Gigs.tsx              # all gigs moderation table
|   |   | - Reviews.tsx           # review moderation
|   |   | - Universities.tsx      # add/manage universities
|   |   | - AuditLog.tsx          # immutable action log viewer
|   |   | - NotFound.tsx
|   | - hooks/                    # useAdminAuth, useUsers, useGigs, useAuditLog
|   | - services/
|   |   | - adminApi.ts           # Axios instance pointing to /api/v1/superadmin
|   |   | - adminAuth.service.ts  # login(), logout(), getMe()
```

```

| |- store/
| | ` - adminAuthStore.ts    # { admin, token, setAdmin, clearAdmin }
| |- types/                  # AdminUser, AuditLog, PlatformStats
| |- App.tsx
| | ` - main.tsx
|- .env.example              # VITE_API_URL=http://localhost:5000/api/v1
|- tailwind.config.ts
|- tsconfig.json
| ` - vite.config.ts

```

4.4 Frontend structure

```

frontend/src/
|- components/
| |- ui/          # shadcn/ui re-exports
| |- gigs/        # GigCard, GigBadge, GigFilters
| |- layout/      # Navbar, Sidebar, Footer
| | ` - shared/   # Avatar, Rating, PageLoader
|- pages/
| |- Landing.tsx
| |- Auth/        # Login.tsx Register.tsx
| |- Gigs/        # Feed.tsx Detail.tsx Create.tsx
| |- Dashboard/   # WorkerDashboard.tsx PosterDashboard.tsx
| | ` - Profile/  # Profile.tsx EditProfile.tsx
|- hooks/  |- services/  |- store/  |- types/
|- utils/  |- styles/    ` - main.tsx

```

4.5 Coding conventions

Convention	Rule
Variables / functions	camelCase e.g. getUserById, gigStatus
Classes / interfaces	PascalCase e.g. GigService, CreateGigDto

Constants	UPPER_SNAKE_CASE e.g. SUPERADMIN_JWT_SECRET
React component files	PascalCase e.g. GigCard.tsx
Service / util files	camelCase e.g. superadmin.service.ts
DB tables	snake_case plural e.g. super_admins, audit_logs
API endpoint paths	kebab-case e.g. /api/v1/superadmin/audit-logs
Commit messages	Conventional Commits: feat: fix: docs: test: refactor: chore:
TypeScript strictness	strict: true in all tsconfig files. No implicit any.

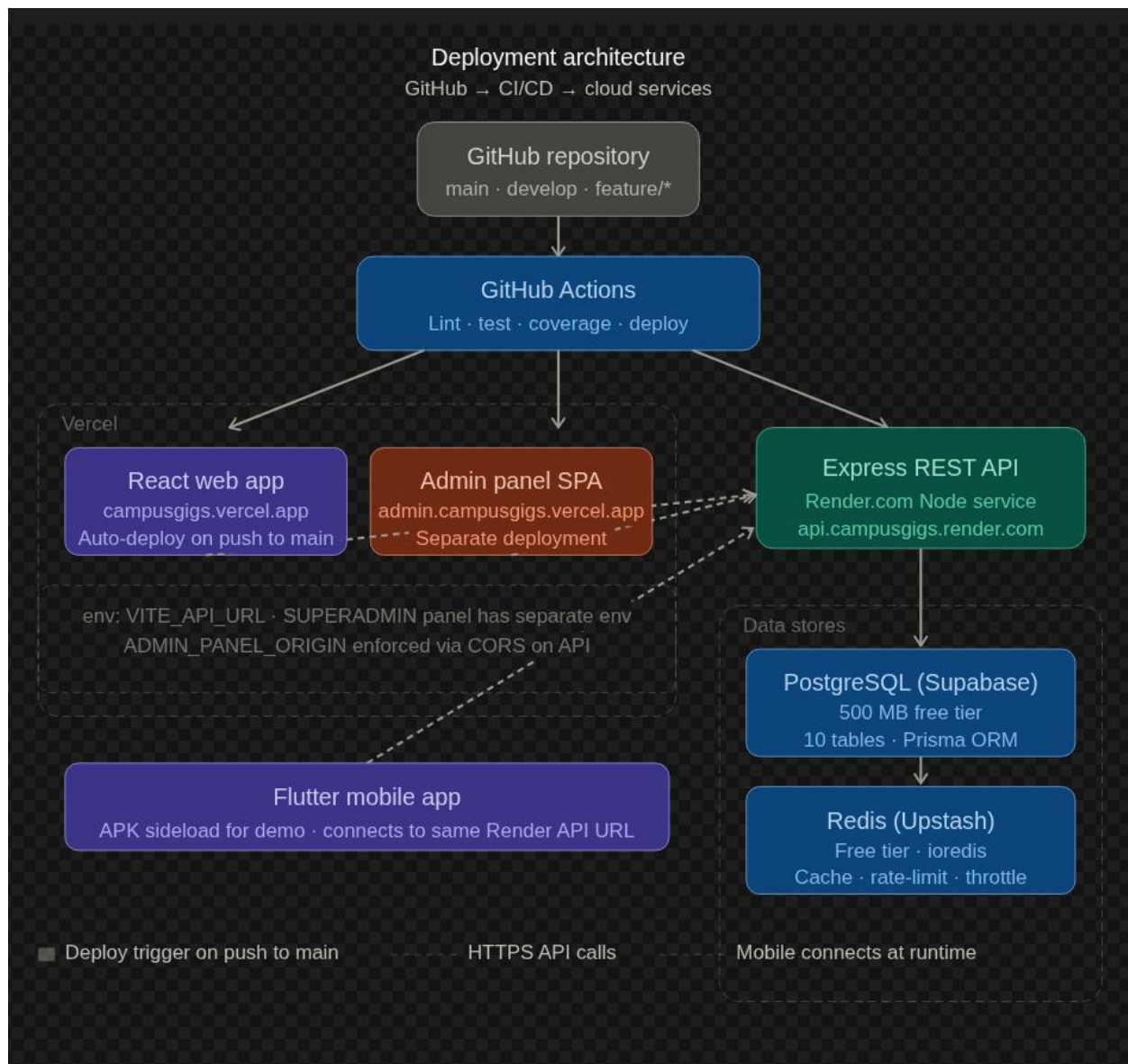


Figure 6: Deployment Architecture

5. GIT REPOSITORY AND BRANCHING STRATEGY

5.1 Branching model

Branch	Protected ?	Purpose
main	<i>Yes (PR required)</i>	Production-ready. Merged from develop via PR. Auto-deploys all three apps (frontend, backend, admin-panel).
develop	<i>Yes (PR required)</i>	Integration branch. All features merge here first.
web	<i>Yes (PR required)</i>	The branch that has the entire web app frontend
server	<i>Yes (PR required)</i>	The branch that contains the entire application API
mobile	<i>Yes (PR required)</i>	The branch that contains the mobile app.
integration	<i>Yes (PR required)</i>	The branch that contains the web app, which is completely wired to the API
super_admin	<i>Yes (PR required)</i>	The branch that contains the super admin web app
feature/*	<i>No</i>	Feature work. e.g. feature/superadmin-auth, feature/audit-log
bugfix/*	<i>No</i>	Bug fixes from develop.
hotfix/*	<i>No</i>	Critical prod fixes from main. Merged to both main and develop.

5.2 Commit message standard (Conventional Commits)

```
feat: add super admin credentials login endpoint
feat: add audit log write on every superadmin mutation
feat: add admin-panel Dashboard page with platform stats
fix: resolve N+1 query on gig feed endpoint
docs: update API spec with /superadmin endpoints
test: add unit tests for superadmin.auth bcrypt verification
chore: add SUPERADMIN_JWT_SECRET to env.example
```

5.3 Pull request checklist

PR description states: what changed, why, and how it was tested

All existing tests pass (CI green)

New code has accompanying unit or integration tests

No new ESLint or TypeScript errors

Swagger/OpenAPI doc updated if a new endpoint was added

superadminGuard is applied on ALL /superadmin/* routes — verify before merge

No .env secrets committed (GitHub secret scanner enabled)

5.4 GitHub repository requirements

README.md with project description, setup guide, env vars, tech stack, demo URLs

STANDUP_LOG.md — daily stand-up log

.gitignore configured for Node.js, React, Flutter, and Docker

GitHub Projects board: Backlog | To Do | In Progress | Review | Done

GitHub Issues for every feature and bug, linked to PRs on completion

Branch protection rules: require PR + CI checks on main and develop

GitHub Actions: lint + test on every PR, deploy all three apps on merge to main

Release tags at end of each phase: v0.1, v0.2, v1.0

5.5 Environment variables

Variable	Purpose
DATABASE_URL	PostgreSQL connection string
REDIS_URL	Upstash or Render Redis URL
BETTER_AUTH_SECRET	Better Auth JWT signing secret (students/posters)
GOOGLE_CLIENT_ID / SECRET	Google OAuth credentials for Better Auth
CLOUDINARY_URL	Cloudinary SDK connection string
RESEND_API_KEY	Resend transactional email API key
SUPERADMIN_JWT_SECRET	Separate JWT secret for super admin tokens. Must be at least 64 chars. Rotate independently of BETTER_AUTH_SECRET.
SUPERADMIN_EMAIL	Default super admin email seeded on first deploy.
SUPERADMIN_PASSWORD	Default super admin password. Must be changed after first login. Never commit to git.
ADMIN_PANEL_ORIGIN	CORS origin for the admin panel SPA. Separate from student frontend origin.